

KR MANGALAM UNIVERSITY

Web Development – III

Unit 2: Express.js, MVC Architecture & REST APIs

Lectures 9 – 18

Lecture Notes

Prepared By – Pratik Raj Verma

Technical Trainer, SOET

| Table of Contents

Lecture	Topic
Lecture 9	Introduction to Express.js & Setup
Lecture 10	Basic CRUD Operations Using Arrays (Without MVC)
Lecture 11	Request & Response Handling in Express.js
Lecture 12	MVC Architecture (Model–View–Controller)
Lecture 13	Controllers & Basic Backend Logic
Lecture 14	CRUD Operations using MVC Pattern
Lecture 15	Express Routing & REST APIs
Lecture 16	Middleware Basics
Lecture 17	Middleware Chaining
Lecture 18	Error Handling in Express

Lecture 9

Introduction to Express.js & Setup

Learning Objectives

- Understand what Express.js is
- Understand why Express.js is used instead of the Node.js HTTP module
- Install Express.js in a project
- Create and run a basic Express server
- Create simple routes
- Access routes using localhost

What is Express.js?

Express.js is a fast, lightweight, and popular web framework built on top of Node.js. It is used to create web servers and APIs quickly and efficiently. Node.js can create servers on its own, but Express.js provides additional features that make backend development easier and more organized.

Features of Express.js

- Simple Routing
- Middleware Support
- Easy Request and Response Handling
- Faster API Development
- Scalable Project Structure

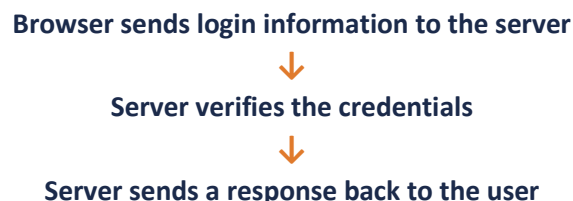
Real-World Example

Applications such as Instagram, Facebook, YouTube, and Netflix use backend servers to handle requests and send data to users. Express.js helps developers create such backend systems efficiently.

Why Do We Need a Backend Server?

- User Authentication
- Database Communication
- Sending and Receiving Data
- Processing Business Logic
- Managing APIs

Example — Instagram Login



Why Use Express.js Instead of the Node HTTP Module?

Using Node's HTTP Module

```
const http = require("http");

const server = http.createServer((req, res) => {
  res.end("Hello World");
});

server.listen(3000);
```

Using Express.js

```
const express = require("express");
const app = express();

app.get("/", (req, res) => {
  res.send("Hello World");
});

app.listen(3000);
```

Node HTTP	Express.js
More Code	Less Code
Manual Routing	Simple Routing
Difficult to Scale	Easy to Scale
Beginner Friendly	Highly Beginner Friendly

Installing Express.js

Step 1 — Create a Project Folder

```
mkdir express-app
```

Step 2 — Open the Project in VS Code

```
code .
```

Step 3 — Initialize a Node.js Project

```
npm init -y
```

npm stands for Node Package Manager. It is used to install and manage packages in Node.js projects.

Step 4 — Install Express.js

```
npm install express
```

```
# Shortcut  
npm i express
```

Project Structure

```
express-app/  
├── node_modules/  
├── package.json  
├── package-lock.json  
└── index.js
```

Understanding Important Files

File	Purpose
node_modules	Contains all installed packages and dependencies (e.g., Express, Mongoose, Nodemon, JWT)
package.json	Stores project name, version, scripts, and dependencies
package-lock.json	Maintains exact versions of installed packages
index.js	Main entry point of the application — the server starts from here

Creating the First Express Server

```
const express = require("express");  
const app = express();  
  
app.get("/", (req, res) => {  
  res.send("Hello World");  
});  
  
app.listen(3000, () => {  
  console.log("Server Running");  
});
```

Understanding the Code

- `const express = require("express")` — imports the Express package
- `const app = express()` — creates an Express application object used to define routes and configure the server
- `app.get()` creates a GET route; `/` represents the homepage route
- `req` represents the incoming request, `res` represents the outgoing response
- `res.send()` sends data back to the client

Request & Response — Quick Look

Concept	Description
Request (req)	Sent by the client to the server — e.g., GET /
Response (res)	Sent by the server back to the client — e.g., "Hello World"

Understanding app.listen()

```
app.listen(3000, () => {
  console.log("Server Running");
});

// Output: Server Running
```

What is localhost?

localhost refers to your own computer acting as a server. When a server runs on port 3000, it can be accessed at <http://localhost:3000>. The browser sends requests directly to your computer.

What is a Port?

A port is a communication endpoint used by applications.

Real-World Analogy

Apartment Building Analogy	Computer/Networking
Building	Computer
Apartment Number	Port Number

Different applications use different ports — common examples: 3000, 5000, 8000, 8080.

Creating Multiple Routes

```
app.get("/", (req, res) => {
  res.send("Home Page");
});

app.get("/about", (req, res) => {
  res.send("About Page");
});

app.get("/contact", (req, res) => {
  res.send("Contact Page");
});
```

Running the Server

```
node index.js
```

```
# Open browser
http://localhost:3000
```

Output

URL	Output
localhost:3000	Home Page
localhost:3000/about	About Page
localhost:3000/contact	Contact Page

Common Errors

Error	Solution
Cannot find module 'express'	Run: npm install express
Port already in use	Change the port number, e.g. app.listen(5000)

Practical Exercise

Create the following routes: /, /about, /contact, /courses

Expected Output: Home Page, About Page, Contact Page, Courses Page

Homework

- Task 1: Create routes for /, /about, /contact, /courses
- Task 2: Change the port number from 3000 to 5000
- Task 3: Customize the response text — `res.send("Welcome to My Server")`

Summary

- Express.js is a backend framework built on Node.js that simplifies server and API development.
- Key concepts: Express basics, installation, project structure, servers, routes, req/res, localhost, ports.
- Express.js forms the foundation of modern MERN Stack development.

Lecture 10

Basic CRUD Operations Using Arrays (Without MVC)

Learning Objectives

- Understand CRUD Operations
- Understand how data can be stored in arrays
- Create APIs using Express.js
- Perform Create, Read, Update, and Delete operations
- Test APIs using Browser and Postman

What is CRUD?

Operation	Meaning
Create	Add New Data
Read	View Existing Data
Update	Modify Existing Data
Delete	Remove Existing Data

CRUD operations are the foundation of almost every application — e.g., Student Management Systems, Employee Management Systems, E-commerce Applications, Social Media Applications, Banking Systems.

Real-World Example — Attendance Register

- Create — Adding a new student's name (Pratik, Rahul, Aman)
- Read — Viewing all student names
- Update — Changing "Rahul" to "Rahul Sharma"
- Delete — Removing a student from the register

Why Learn CRUD Using Arrays First?

Before connecting a database, it is important to understand how CRUD logic works. Using arrays helps students focus on API Development, Request Handling, Route Creation, and CRUD Logic — without worrying about databases.

Creating a Simple Student Management API

Step 1 — Install Express

```
npm init -y
npm install express
```

Step 2 — Project Structure

```
student-crud/
|
```



```
├─ node_modules/  
├─ package.json  
└─ index.js
```

Creating an Express Server

```
const express = require("express");  
const app = express();  
  
app.listen(3000, () => {  
  console.log("Server Running");  
});
```

Understanding Student Data

Instead of a database, we use an array to store data temporarily:

```
let students = ["Pratik", "Rahul", "Aman"];
```

What is an API?

API stands for Application Programming Interface. An API allows communication between Frontend and Backend — e.g., React ↔ Express, Mobile App ↔ Express, Website ↔ Server.

Read Operation

Goal: Display all students.

```
app.get("/students", (req, res) => {  
  res.send(students);  
});
```

Browser Request → /students



Server



Students Array



Response

URL: <http://localhost:3000/students>

Create Operation

Goal: Add a new student.

Middleware Required

Before receiving data from Postman, Express must understand JSON:

```
app.use(express.json());
```

Route

```
app.post("/students", (req, res) => {
  const studentName = req.body.name;
  students.push(studentName);
  res.send("Student Added Successfully");
});
```

Request Body:

```
{ "name": "Rohit" }
```

Update Operation

Goal: Update an existing student name.

```
app.put("/students/:index", (req, res) => {
  const index = req.params.index;
  const newName = req.body.name;
  students[index] = newName;
  res.send("Student Updated Successfully");
});
```

URL: <http://localhost:3000/students/1> Request Body: { "name": "Rahul Sharma" }

Delete Operation

Goal: Remove a student.

```
app.delete("/students/:index", (req, res) => {
  const index = req.params.index;
  students.splice(index, 1);
  res.send("Student Deleted Successfully");
});
```

Understanding splice(): array.splice(startIndex, deleteCount) — e.g., students.splice(1, 1) starts deleting from index 1 and removes 1 element.

Complete CRUD Program

```
const express = require("express");
const app = express();
app.use(express.json());

let students = ["Pratik", "Rahul", "Aman"];

// Read
app.get("/students", (req, res) => { res.send(students); });

// Create
app.post("/students", (req, res) => {
  students.push(req.body.name);
  res.send("Student Added Successfully");
});
```

```
// Update
app.put("/students/:index", (req, res) => {
  students[req.params.index] = req.body.name;
  res.send("Student Updated Successfully");
});

// Delete
app.delete("/students/:index", (req, res) => {
  students.splice(req.params.index, 1);
  res.send("Student Deleted Successfully");
});

app.listen(3000, () => { console.log("Server Running"); });
```

Testing APIs Using Postman

API	Method	URL / Body
Read	GET	http://localhost:3000/students
Create	POST	http://localhost:3000/students Body: { "name": "Rohit" }
Update	PUT	http://localhost:3000/students/1 Body: { "name": "Rahul Sharma" }
Delete	DELETE	http://localhost:3000/students/1

Common Errors

Error	Reason	Solution
Cannot Read Property 'name'	Data not sent in request body	Send JSON data properly
req.body is undefined	JSON middleware missing	app.use(express.json())
Student Not Updated	Incorrect index provided	Check URL parameter

Practical Exercise & Homework

Create CRUD APIs for: ["Aman", "Rahul", "Rohit"] — Create, Read, Update, Delete Student APIs, tested using Postman.

- Homework Task 1: Create CRUD APIs using ["Pratik", "Rahul", "Aman"]
- Homework Task 2: Add at least 5 students using Postman
- Homework Task 3: Update any student name
- Homework Task 4: Delete any student from the array

Summary

- CRUD stands for Create, Read, Update, and Delete.
- Key concepts: Arrays in JavaScript, Express Routes, GET/POST/PUT/DELETE, req.body, req.params, express.json(), Postman Testing.

- This lecture builds the foundation for backend development — future lectures cover objects, MVC architecture, and databases like MongoDB.

Lecture 11

Request & Response Handling in Express.js

Learning Objectives

- Understand the Request-Response Cycle
- Understand the Request Object (req) and Response Object (res)
- Work with req.body, req.params, req.query, and req.headers
- Send responses using res.send(), res.json(), and res.status()
- Understand the purpose of express.json()
- Test APIs using Postman

Introduction

Whenever a user interacts with a website or application — login forms, registration forms, search, product listings, student management systems — communication happens between client and server. This is called the Request-Response Cycle.

What is a Request?

A Request is a message sent by the client (browser, mobile app, Postman, frontend application) to the server asking for some action. Example: GET /students means the client is requesting student data; POST /students means the client wants to add a new student.

What is a Response?

A Response is the reply sent by the server back to the client. Example — Request: GET /students → Response: [{ "name": "Rahul" }].

Real-World Analogy — Restaurant

Restaurant	Backend
Customer	Client
Order	Request
Waiter	Server
Food	Response

Request-Response Cycle





Understanding the Request Object (req)

Whenever a request reaches the server, Express creates a Request Object (accessible as req in the handler (req, res)). It contains URL, Method, Parameters, Query Strings, Headers, Body Data, and more.

Important Properties of req

- req.body
- req.params
- req.query
- req.headers

req.body

Contains data sent by the client in the request body. Usually used with POST, PUT, PATCH requests.

```
app.post("/students", (req, res) => {
  console.log(req.body);
  res.send("Student Added");
});
```

Why is express.json() Required?

Express cannot read JSON data automatically. We must enable this middleware:

```
app.use(express.json());
```

Without this middleware, req.body returns undefined.

req.params

Route Parameters are dynamic values present in the URL.

```
app.get("/students/:id", (req, res) => {
  const id = req.params.id;
  res.send(id);
});
```

Request: GET /students/101 → Response: 101

Multiple Parameters

```
app.get("/students/:id/course/:course", (req, res) => {
  res.json(req.params);
});

// Request: GET /students/101/course/MERN
// Response: { "id":"101", "course":"MERN" }
```

req.query

Query Parameters are values passed after a question mark (?) in the URL.

```
app.get("/students", (req, res) => {
  const course = req.query.course;
  res.send(course);
});

// Request: GET /students?course=MERN → Response: MERN
```

Real-World Example — E-commerce

- Filter products: /products?category=mobile
- Search products: /products?search=iphone
- Sort products: /products?sort=price

req.params vs req.query

Feature	req.params	req.query
Location	URL Path	URL Query String
Required	Usually Yes	Usually Optional
Example	/students/101	/students?id=101

req.headers

Headers contain additional information about the request — e.g., Content-Type, Authorization Token, Browser Information, Language Settings.

```
const token = req.headers.authorization;
```

Used heavily in Authentication and JWT. After login, every future request sends: Authorization: Bearer xyz123 through headers.

Understanding the Response Object (res)

The Response Object is used to send data back to the client. Common methods: `res.send()`, `res.json()`, `res.status()`.

res.send()

Sends text, HTML, or data back to the client.

```
app.get("/", (req, res) => { res.send("Hello World"); });
```

res.json()

Used to send JSON data — the most common method in APIs.

```
app.get("/student", (req, res) => {
  res.json({ name: "Rahul", course: "MERN" });
});
```

JSON is lightweight, easy to read, easy to parse, and the industry standard — almost every API uses JSON.

res.status()

Used to send HTTP Status Codes.

```
res.status(200).send("Success");
```

Status Code	Meaning
200	Success
201	Resource Created
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
500	Internal Server Error

Chaining Methods

```
res.status(200).json({ message: "Success" });
```

Practical — Student API

```
// Create Student
app.post("/students", (req, res) => {
  const student = req.body;
  res.status(201).json({ message: "Student Added", student });
});

// Get Student By ID
app.get("/students/:id", (req, res) => {
  res.json({ id: req.params.id });
});

// Search Student
app.get("/students", (req, res) => {
  res.json({ course: req.query.course });
});
```

Common Mistakes

Mistake	Fix
Forgetting <code>express.json()</code> → <code>req.body</code> is undefined	<code>app.use(express.json());</code>
Wrong parameter name (route <code>/students/:id</code> but using <code>req.params.studentId</code>)	Use <code>req.params.id</code> to match the route
Forgetting status codes (<code>res.send</code> only)	Use <code>res.status(201).send(...)</code>

Interview Questions

What is the Request Object?

The Request Object (req) contains information sent by the client to the server.

What is the Response Object?

The Response Object (res) is used to send data back to the client.

What is req.body?

It contains data sent in the request body.

What is req.params?

It contains route parameters from the URL.

What is req.query?

It contains query string values from the URL.

Why do we use express.json()?

It converts incoming JSON data into a JavaScript object so it can be accessed using req.body.

What is res.json()?

It sends JSON data to the client.

Summary

- Request and Response Handling is the foundation of backend development.
- Key concepts: Request-Response Cycle, req/res objects, req.body, req.params, req.query, req.headers, express.json(), res.send()/res.json()/res.status().
- These concepts are used throughout the MERN Stack journey.

Lecture 12

MVC Architecture (Model–View–Controller)

Introduction

As applications grow, managing all code in a single file becomes difficult. Imagine a Notes Application with hundreds of APIs, database operations, authentication, and business logic — if everything is in one file, the project becomes hard to understand, debug, and maintain. MVC Architecture solves this problem.

MVC stands for: M – Model, V – View, C – Controller. It is a design pattern that organizes code by separating responsibilities into different files and folders.

Why Do We Need MVC?

```
app.get("/notes", async (req, res) => {  
  const notes = await Note.find();  
  res.json(notes);  
});
```

This works fine for small applications. But real-world projects contain 100+ APIs, Authentication, User Management, Products, Orders, Payments, Admin Dashboards. If all code sits in one file, the project becomes messy and hard to maintain. MVC solves this by dividing the application into layers.

Real-World Analogy — Restaurant

- Step 1: Customer places an order (requests a burger)
- Step 2: Waiter receives the order and communicates with the kitchen
- Step 3: Chef prepares the food
- Step 4: Waiter delivers the food back to the customer

Restaurant	MVC
Customer	Client / User
Waiter	Controller
Chef	Model
Kitchen	Database Storage
Delivered Food	Response

The Controller acts as a middleman between the user and the database.

Components of MVC

1. Model

The Model is responsible for managing data and interacting with the database. It defines data structure, database schema, and database operations.

```
import mongoose from "mongoose";
```

```
const noteSchema = new mongoose.Schema({
  title: String,
  content: String
});

export default mongoose.model("Note", noteSchema);
```

Real-World Example: a student admission form collects Name, Email, Phone, Course. The Model defines how this data should be stored — it acts as a blueprint for data.

2. Controller

A Controller contains the business logic of the application: receiving requests, processing data, communicating with models, and sending responses.

```
import Note from "../models/Note.js";

export const getNotes = async (req, res) => {
  const notes = await Note.find();
  res.json(notes);
};
```

Analogy: the waiter receives the order, talks to the kitchen, and returns the food — similarly, a Controller receives the request, talks to the database through the model, and returns the response.

3. Route

Routes define which URL should execute which controller.

```
router.get("/notes", getNotes);
```

Analogy: a route is like a road sign — it tells requests where to go.

4. Database

The database stores application data permanently. Popular databases include MongoDB, MySQL, and PostgreSQL. In the MERN Stack, MongoDB is commonly used.

5. View

The View displays information to users. In traditional MVC this means HTML, CSS, and Templates. In MERN Stack applications, React acts as the View layer — so backend development mainly focuses on Models, Controllers, Routes, and the Database, while the frontend handles the View.

MVC Folder Structure

```
project/
├── server.js
├── config/
│   └── db.js
├── models/
│   └── Note.js
└── controllers/
```

```
├── noteController.js
├── routes/
│   └── noteRoutes.js
└── package.json
```

Understanding Each Folder

Config Folder

Contains configuration files: database connection, JWT configuration, Cloudinary configuration, environment settings.

```
import mongoose from "mongoose";

const connectDB = async () => {
  await mongoose.connect(process.env.MONGO_URI);
  console.log("Database Connected");
};

export default connectDB;
```

Models Folder

Contains all database schemas — models define how data is stored.

Controllers Folder

Contains all business logic — controllers decide what should happen when a request arrives.

Routes Folder

Contains API endpoints — routes decide which controller should handle the request.

```
router.get("/", getNotes);
router.post("/", createNote);
router.delete("/:id", deleteNote);
```

server.js

The entry point of the application — the server starts from this file.

Request Flow in MVC

Suppose a user requests GET /api/notes:



Database



Model



Controller



Response

Separation of Concerns

One of MVC's biggest advantages is Separation of Concerns — every file should have only one responsibility.

Bad Practice

```
app.get("/notes", async (req, res) => {  
  const notes = await Note.find();  
  res.json(notes);  
});  
// Route, logic, and database operation all mixed together
```

Good Practice

```
// Route  
router.get("/", getNotes);  
  
// Controller  
export const getNotes = async (req, res) => {  
  const notes = await Note.find();  
  res.json(notes);  
};  
  
// Model  
const noteSchema = new mongoose.Schema({  
  title: String,  
  content: String  
});
```

Practical Example — Notes API Using MVC

Note Model

```
import mongoose from "mongoose";  
  
const noteSchema = new mongoose.Schema({  
  title: { type: String, required: true },  
  content: { type: String, required: true }  
});  
  
export default mongoose.model("Note", noteSchema);
```

Controller

```
import Note from "../models/Note.js";

export const createNote = async (req, res) => {
  const note = await Note.create(req.body);
  res.status(201).json(note);
};
```

Route

```
import express from "express";
import { createNote } from "../controllers/noteController.js";

const router = express.Router();
router.post("/", createNote);

export default router;
```

Server

```
app.use("/api/notes", noteRoutes);
```

Testing Using Postman

Request	Body / Response
POST /api/notes	{ "title": "MVC Notes", "content": "Learning MVC Architecture" }
Response	{ "_id": "123", "title": "MVC Notes", "content": "Learning MVC Architecture" }

Industry Example — E-Commerce Application

Layer	Examples
Models	User, Product, Cart, Order, Payment
Controllers	Login User, Register User, Create Order, Cancel Order, Add Product
Routes	/api/users, /api/products, /api/orders, /api/payments

Without MVC, everything becomes difficult to manage. With MVC: code remains organized, bugs are easier to find, team collaboration becomes easier, and new features can be added quickly.

Advantages of MVC

- Clean and Organized Code
- Easy Debugging
- Easy Maintenance
- Better Team Collaboration
- Improved Scalability
- Reusable Components
- Industry Standard Architecture

Interview Questions

What is MVC Architecture?

MVC is a software design pattern that separates an application into Model, View, and Controller components to improve organization, maintainability, and scalability.

What is the role of a Controller?

A Controller handles requests, processes business logic, interacts with Models, and sends responses.

What is a Model?

A Model represents the structure of data and manages database operations.

Why is MVC used?

MVC keeps code organized, maintainable, scalable, and easy to understand.

What is Separation of Concerns?

Separation of Concerns means dividing an application into different sections where each section has only one responsibility.

Summary

- Model → Handles Data and Database Operations
- View → Displays Information to Users
- Controller → Handles Business Logic and Requests
- Route → Connects URLs with Controllers
- Database → Stores Data Permanently
- MVC is widely used in Node.js, Express.js, Laravel, Django, Spring Boot, and many other frameworks.

Lecture 13

Controllers & Basic Backend Logic

Learning Objectives

- Understand the role of Controllers in Backend Development
- Understand why Controllers are called the "Brain" of the Backend
- Write functions to handle application logic
- Store and manipulate data using arrays
- Use JavaScript methods such as push(), find(), filter(), and splice()
- Build simple APIs using Controllers
- Understand the flow between Route and Controller

Introduction

In the previous lecture, we learned how to create routes using Express.js. Writing all logic directly inside routes is not good practice — as applications grow, routes become cluttered with business logic, making the code difficult to manage. To solve this, we use Controllers.

What is a Controller?

A Controller is a JavaScript function that contains the business logic of the application. It receives a request from the client, processes the data, and sends a response back. In simple words: a Controller decides what should happen when a user makes a request.

Real-World Analogy — Restaurant

Restaurant	Backend
Customer	Client
Waiter	Controller
Kitchen	Business Logic
Food	Response

Why Do We Need Controllers?

```
// Without Controllers
app.get("/students", (req, res) => {
  const students = ["Rahul", "Amit", "Priya"];
  res.json(students);
});
```

This works for small projects, but imagine 100 routes, Authentication, Payments, Orders, and Products — everything inside routes becomes difficult to manage. Therefore, we move the logic into Controllers.

Controller as the Brain of Backend

The Controller is often called the brain of the backend because it receives requests, processes data, validates information, calls models, performs calculations, and sends responses.

Example — User Login

- 1. Receives email and password
- 2. Checks whether the user exists
- 3. Verifies the password
- 4. Generates a token
- 5. Sends the response

Backend Flow



When databases are introduced later, the flow extends to:



Creating a Controller

studentController.js

```
export const getStudents = (req, res) => {  
  res.send("Students Data");  
};
```

Connecting Controller with Route

```
import express from "express";  
import { getStudents } from "../controllers/studentController.js";  
  
const router = express.Router();  
router.get("/", getStudents);  
  
export default router;
```

Now the route delegates the work to the Controller.

Functions in Backend

Controllers are simply JavaScript functions. A normal function performs a task; a Controller function handles requests and responses.

```
function greet() { console.log("Hello"); }

// Controller Example
export const getStudents = (req, res) => {
  res.send("Student List");
};
```

Array-Based Data Handling

Before learning databases, we use arrays to simulate data storage — think of the array as a temporary database.

```
let students = ["Rahul", "Amit", "Priya"];
```

What is Data Manipulation?

Operation	Meaning
Create	Add Data
Read	Get Data
Update	Modify Data
Delete	Remove Data

Common Mistakes

Mistake	Fix
Forgetting to export a controller	Use <code>export const getStudents = () => {};</code>
Forgetting <code>express.json()</code>	<code>app.use(express.json());</code> — without it, <code>req.body</code> is undefined

Interview Questions

What is a Controller?

A Controller is a function that handles requests, executes business logic, and sends responses.

Why is a Controller called the Brain of the Backend?

Because it contains the application's logic and decides how requests should be processed.

Summary

- Controllers are responsible for handling application logic.
- Key concepts: What is a Controller, Controller Working Flow, Backend Logic Creation.
- Controllers help keep routes clean and make backend applications easier to manage and scale — they are one of the most important parts of MVC Architecture.

Lecture 14

CRUD Operations using MVC Pattern

Student Data

This array acts as a temporary database. All CRUD operations will be performed on this array.

```
let students = [  
  { id: 1, name: "Pratik", age: 25 },  
  { id: 2, name: "Rahul", age: 22 },  
];
```

CREATE Operation

Create operation is used to add new data into the application.

Controller Logic

```
const createStudent = (req, res) => {  
  const student = req.body;  
  students.push(student);  
  res.send("Student Added Successfully");  
};
```

- req.body contains student data sent by the client
- The student object is added to the array using push()
- A success response is returned to the client

Route

```
app.post("/students", createStudent);
```

READ Operation

Read operation is used to retrieve data from the application.

Controller Logic

```
const getStudents = (req, res) => {  
  res.send(students);  
};
```

Route

```
app.get("/students", getStudents);
```

READ Single Student — Get Student By ID

```
const getStudentById = (req, res) => {  
  const id = Number(req.params.id);
```

```
const student = students.find((student) => student.id === id);
res.send(student);
};
```

- Student ID is received through URL parameters
- find() searches for the matching student
- The matched student object is returned

Route

```
app.get("/students/:id", getStudentById);
```

UPDATE Operation

Update operation is used to modify existing records.

```
const updateStudent = (req, res) => {
  const id = Number(req.params.id);
  const student = students.find((student) => student.id === id);

  if (!student) {
    return res.send("Student Not Found");
  }

  student.name = req.body.name;
  student.age = req.body.age;

  res.send("Student Updated Successfully");
};
```

Route

```
app.put("/students/:id", updateStudent);
```

DELETE Operation

Delete operation is used to remove data from the application.

```
const deleteStudent = (req, res) => {
  const id = Number(req.params.id);
  const index = students.findIndex((student) => student.id === id);

  if (index === -1) {
    return res.send("Student Not Found");
  }

  students.splice(index, 1);
  res.send("Student Deleted Successfully");
};
```

- findIndex() finds the position of the student
- splice() removes the student from the array

Route

```
app.delete("/students/:id", deleteStudent);
```

SEARCH Operation — Search Student By Name

```
const searchStudent = (req, res) => {  
  const name = req.query.name;  
  const student = students.find(  
    (student) => student.name.toLowerCase() === name.toLowerCase()  
  );  
  res.send(student);  
};
```

Route

```
app.get("/search", searchStudent);  
  
// Example URL: /search?name=rahul
```

FILTER Operation — Filter Students By Age

```
const filterStudents = (req, res) => {  
  const age = Number(req.query.age);  
  const filteredStudents = students.filter((student) => student.age > age);  
  res.send(filteredStudents);  
};
```

Route

```
app.get("/filter", filterStudents);  
  
// Example URL: /filter?age=21
```

Exporting Controllers

After creating all controller functions, export them so they can be used inside routes.

```
module.exports = {  
  createStudent,  
  getStudents,  
  getStudentById,  
  updateStudent,  
  deleteStudent,  
  searchStudent,  
  filterStudents,  
};
```

Why Do We Export Controllers?

- To keep business logic separate from routes
- To follow MVC architecture

- To make the project more organized and scalable
- Routes only handle endpoints, while controllers handle logic

Lecture 15

Express Routing & REST APIs

Learning Objectives

- Understand Routing in Express.js
- Understand API Endpoints
- Work with GET, POST, PUT, and DELETE methods
- Understand REST APIs and REST principles
- Design RESTful API Endpoints
- Understand Static and Dynamic Routes
- Work with Route Parameters
- Use Express Router and implement Modular Routing
- Connect Routes with Controllers
- Follow REST API Naming Conventions

Introduction

In modern web applications, the frontend communicates with the backend using APIs. Whenever a client sends a request, the server must decide which URL was requested, which controller should execute, and what response should be returned. This is achieved using Routing and REST APIs.

What is Routing?

Routing is the process of defining how an application responds to client requests for specific URLs and HTTP methods.

```
app.get("/students", (req, res) => {
  res.send("All Students");
});
```

Real-World Analogy — University

Department	Responsibility
Admission	Student Admission
Accounts	Fee Management
Library	Books
Examination	Results
URL	Responsibility
/students	Student Data
/teachers	Teacher Data

Department	Responsibility
/courses	Course Data

What is an API Endpoint?

An API Endpoint is a URL where a client sends requests — e.g., /students, /teachers, /courses, /users. Each endpoint performs a specific task; GET /students returns all students.

What is a REST API?

REST stands for Representational State Transfer. A REST API follows standard rules for creating APIs, using Resources, HTTP Methods, and Meaningful URLs.

REST Principles

- Use resources as URLs
- Use HTTP methods properly
- Follow consistent naming conventions
- Be easy to understand and maintain

Resource-Based Routing

Bad Example	Good Example
/getStudents, /createStudent, /deleteStudent	/students (use HTTP methods to decide the action)

Understanding HTTP Methods

Method	Purpose
GET	Retrieve Data
POST	Create Data
PUT	Update Data
DELETE	Remove Data

CRUD and HTTP Methods

CRUD Operation	HTTP Method
Create	POST
Read	GET
Update	PUT
Delete	DELETE

Route Examples by Method

GET Route

```
app.get("/students", (req, res) => {  
  res.send("All Students");  
});  
// Request: GET /students
```

POST Route

```
app.post("/students", (req, res) => {  
  res.send("Student Added");  
});  
// Request: POST /students
```

PUT Route

```
app.put("/students/:id", (req, res) => {  
  res.send("Student Updated");  
});  
// Request: PUT /students/101
```

DELETE Route

```
app.delete("/students/:id", (req, res) => {  
  res.send("Student Deleted");  
});  
// Request: DELETE /students/101
```

RESTful API Design

Action	Endpoint
Get All Students	GET /students
Get Single Student	GET /students/101
Create Student	POST /students
Update Student	PUT /students/101
Delete Student	DELETE /students/101

REST Naming Conventions

Use Nouns

Good	Bad
/students, /users, /products	/getStudents, /createStudent, /deleteStudent

Use Plural Names

Good	Bad
/students, /courses, /teachers	/student, /course, /teacher

Static vs Dynamic Routes

Static Routes

Static routes have fixed URLs.

```
app.get("/", (req, res) => { res.send("Home Page"); });
app.get("/about", (req, res) => { res.send("About Page"); });
```

Dynamic Routes

Dynamic routes contain variable values.

```
app.get("/students/:id", (req, res) => {
  res.send("Student Details");
});

// Examples: /students/1, /students/10, /students/100
```

Route Parameters

```
app.get("/students/:id", (req, res) => {
  const id = req.params.id;
  res.send(id);
});

// Request: /students/101 → Output: 101
```

Express Router

As projects grow, placing all routes in one file becomes difficult. Express Router helps organize routes.

```
import express from "express";

const router = express.Router();

export default router;
```

Adding Routes

```
router.get("/", getStudents);
router.post("/", createStudent);
router.put("/:id", updateStudent);
router.delete("/:id", deleteStudent);
```

Connecting Router to Server

```
import studentRoutes from "../routes/studentRoutes.js";

app.use("/students", studentRoutes);
```

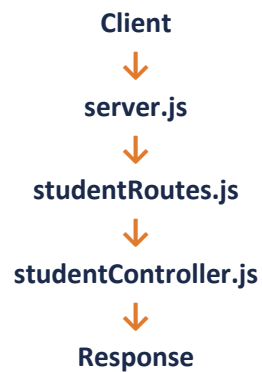
Meaning: all requests starting with /students will be handled by studentRoutes.js.

Modular Routing

Modular Routing means separating routes into different files.

```
project/
├── server.js
├── routes/
│   ├── studentRoutes.js
│   └── teacherRoutes.js
├── controllers/
│   ├── studentController.js
│   └── teacherController.js
└── package.json
```

Request Flow



Practical

Create REST APIs for Students, Teachers, and Courses.

- GET Route
- POST Route
- PUT Route
- DELETE Route
- Express Router
- Modular Routing

Summary

- Key concepts: Routing, API Endpoints, REST API, REST Principles, Resource-Based Routing, HTTP Methods.
- RESTful API Design, REST Naming Conventions, Static & Dynamic Routes, Route Parameters.
- Express Router, `app.use()`, Modular Routing.

- This lecture introduces professional backend API development for real-world Express.js applications.

Lecture 16

Middleware Basics

Learning Objectives

- Understand what Middleware is and why it is used
- Understand the Middleware execution flow
- Understand the purpose of the next() function
- Identify types of Middleware — built-in, custom, and third-party
- Enable CORS in an Express application
- Build a Logger and Authentication Middleware

What is Middleware?

Middleware is a function that sits between the incoming request and the final response. It has access to the request object (req), the response object (res), and a special function called next(). Middleware can inspect, modify, or reject a request before it reaches the route handler.

```
app.use((req, res, next) => {
  console.log("Middleware Running");
  next();
});
```

In simple words: Middleware is code that runs in between the request and the response.

Real-World Analogy — Airport Security

Before a passenger (request) boards a flight (reaches the route handler), they pass through several checkpoints: ticket check, security scan, boarding pass verification. Each checkpoint is a middleware — it checks something and then allows the passenger to move to the next checkpoint.

Airport	Backend
Passenger	Request
Security Checkpoint	Middleware
Boarding Gate	Route Handler
Flight Departure	Response

Why Do We Need Middleware?

Without middleware, every route would need to repeat the same logic — e.g., logging, authentication checks, parsing JSON. Middleware allows this common logic to be written once and reused across many routes.

- Logging requests for debugging
- Authenticating users before granting access
- Parsing incoming JSON or form data
- Validating request data

- Handling errors centrally
- Enabling cross-origin requests (CORS)

Middleware Flow

Every request passes through middleware functions in the order they are defined, before finally reaching the route handler.



The next() Function

`next()` is a function provided by Express that passes control to the next middleware in the chain. If `next()` is not called, the request will hang — the client will never receive a response.

```
app.use((req, res, next) => {
  console.log("Request received");
  next(); // passes control forward
});

app.get("/", (req, res) => {
  res.send("Home Page");
});
```

What Happens Without next()?

```
app.use((req, res, next) => {
  console.log("Request received");
  // next() not called
});

// The request will hang here forever
// The client will never receive a response
```

Types of Middleware

Type	Description
Built-in Middleware	Comes packaged with Express, e.g., <code>express.json()</code> , <code>express.static()</code>
Custom Middleware	Written by the developer for application-specific logic, e.g., a logger or auth check

Type	Description
Third-party Middleware	Installed via npm, e.g., cors, morgan, helmet

1. Built-in Middleware

Express provides some middleware out of the box.

```
app.use(express.json());           // parses incoming JSON
app.use(express.urlencoded());    // parses form data
app.use(express.static("public")); // serves static files
```

2. Custom Middleware

Custom middleware is written by developers to handle specific application needs, such as logging or authentication.

```
const logger = (req, res, next) => {
  console.log(`${req.method} request to ${req.url}`);
  next();
};

app.use(logger);
```

3. Third-party Middleware

Third-party middleware is installed using npm and adds extra functionality without writing it from scratch.

```
npm install cors
npm install morgan
npm install helmet
```

Practical: Logger Middleware

A Logger Middleware records details about every incoming request — useful for debugging.

```
const express = require("express");
const app = express();

const logger = (req, res, next) => {
  const time = new Date().toLocaleTimeString();
  console.log(`[${time}] ${req.method} ${req.url}`);
  next();
};

app.use(logger);

app.get("/", (req, res) => {
  res.send("Home Page");
});

app.listen(3000, () => console.log("Server Running"));
```

Output in terminal when a request is made to "/":


```
[10:15:32] GET /
```

Practical: Authentication Middleware

Authentication Middleware checks whether a user is authorized before allowing access to a route.

```
const authMiddleware = (req, res, next) => {
  const token = req.headers.authorization;

  if (!token) {
    return res.status(401).send("Access Denied: No Token Provided");
  }

  // Token verification logic goes here
  next();
};

app.get("/dashboard", authMiddleware, (req, res) => {
  res.send("Welcome to Dashboard");
});
```

How It Works

- The request first passes through authMiddleware
- If no token is found, the request is rejected with a 401 status
- If a token exists, next() is called and the request proceeds to the route handler

What is CORS?

CORS stands for Cross-Origin Resource Sharing. By default, browsers block requests made from one domain (origin) to a different domain for security reasons. CORS is a mechanism that allows a server to specify which origins are permitted to access its resources.

Real-World Example

A React frontend running on <http://localhost:3000> tries to call an Express backend running on <http://localhost:5000>. Without CORS enabled, the browser blocks this request with a CORS error.

Enabling CORS

```
npm install cors
```

```
const cors = require("cors");
const express = require("express");
const app = express();

app.use(cors());

app.get("/data", (req, res) => {
  res.json({ message: "CORS Enabled Successfully" });
});
```

Restricting CORS to Specific Origins

```
app.use(cors({
  origin: "http://localhost:3000"
}));
```

Common Errors

Error	Reason	Solution
Request hangs indefinitely	next() was not called inside middleware	Always call next() unless sending a response
req.body is undefined	express.json() middleware missing	app.use(express.json());
CORS policy blocked request	CORS not enabled on the server	app.use(cors());

Interview Questions

What is Middleware?

Middleware is a function that has access to req, res, and next, and can execute code, modify the request/response, or end the request-response cycle.

What is the purpose of next()?

next() passes control to the next middleware function in the stack. Without it, the request-response cycle does not complete.

What is the difference between built-in, custom, and third-party middleware?

Built-in middleware ships with Express (e.g., express.json()), custom middleware is written by the developer, and third-party middleware is installed via npm (e.g., cors, morgan).

What is CORS and why is it needed?

CORS (Cross-Origin Resource Sharing) allows a server to control which origins can access its resources, which is necessary because browsers block cross-origin requests by default.

Summary

- Middleware is a function that runs between the request and the response, with access to req, res, and next.
- Middleware types: Built-in (express.json()), Custom (logger, auth check), Third-party (cors, morgan, helmet).
- next() is essential — without it, the request hangs and the client never receives a response.
- CORS allows controlled cross-origin communication between frontend and backend applications.

Lecture 17

Middleware Chaining

Learning Objectives

- Understand how multiple middleware functions execute together
- Understand the Middleware Chaining flow
- Understand the next() sequence across multiple middleware
- Apply Route-level Middleware to specific routes
- Build and execute multiple middleware functions in sequence

Introduction

In the previous lecture, we learned how a single middleware function works. In real applications, however, a request usually needs to pass through multiple middleware functions before reaching the route handler — for example: logging, authentication, and validation, all applied to the same request. This is called Middleware Chaining.

What is Middleware Chaining?

Middleware Chaining means executing multiple middleware functions one after another for the same request, where each middleware calls next() to pass control to the following one.

```
app.use(middleware1);
app.use(middleware2);
app.use(middleware3);

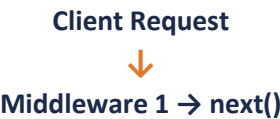
// Execution Order:
// middleware1 -> middleware2 -> middleware3 -> Route Handler
```

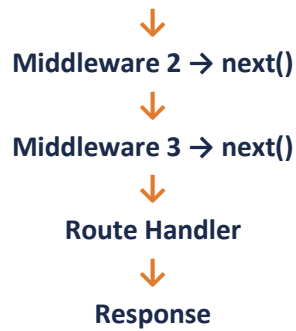
Real-World Analogy — Hospital Visit

When a patient visits a hospital, they pass through several stages in sequence before meeting the doctor: reception desk, nurse for vitals check, then the doctor's cabin. Each stage must be completed before moving to the next — similar to middleware chaining.

Hospital	Backend
Reception Desk	Middleware 1 (Logging)
Nurse (Vitals Check)	Middleware 2 (Authentication)
Doctor's Cabin	Route Handler

Middleware Chaining Flow





Creating Multiple Middleware Functions

```
const express = require("express");
const app = express();

// Middleware 1: Logger
const logger = (req, res, next) => {
  console.log("Logger: Request received");
  next();
};

// Middleware 2: Authentication Check
const authCheck = (req, res, next) => {
  console.log("Auth: Checking token");
  const token = req.headers.authorization;

  if (!token) {
    return res.status(401).send("Unauthorized");
  }
  next();
};

// Middleware 3: Request Time Logger
const requestTime = (req, res, next) => {
  req.requestTime = new Date().toISOString();
  next();
};
```

Applying Middleware Globally

Global middleware runs for every incoming request, regardless of the route.

```
app.use(logger);
app.use(authCheck);
app.use(requestTime);

app.get("/", (req, res) => {
  res.send(`Home Page - Requested at ${req.requestTime}`);
});
```

Execution Order for Every Request



Route-Level Middleware

Instead of applying middleware globally, it can also be applied to specific routes only. This is called Route-level Middleware.

```
app.get("/dashboard", authCheck, (req, res) => {
  res.send("Welcome to Dashboard");
});

app.get("/about", (req, res) => {
  res.send("About Page"); // authCheck does NOT run here
});
```

Here, `authCheck` only runs for the `/dashboard` route — the `/about` route is unaffected.

Chaining Multiple Middleware on a Single Route

```
app.get("/profile", logger, authCheck, (req, res) => {
  res.send("Profile Page");
});

// Execution Order: logger -> authCheck -> Route Handler
```

Middleware functions can also be passed as an array:

```
app.get("/profile", [logger, authCheck], (req, res) => {
  res.send("Profile Page");
});
```

`next()` Sequence in Chaining

Each middleware must call `next()` to hand control over to the following middleware. If any middleware in the chain forgets to call `next()` (and does not send a response), the chain breaks and the request hangs.

```
const middlewareA = (req, res, next) => {
  console.log("A");
  next(); // required to continue to B
};

const middlewareB = (req, res, next) => {
  console.log("B");
  next(); // required to continue to route handler
};
```

```
app.use(middlewareA);
app.use(middlewareB);

app.get("/", (req, res) => {
  res.send("Final Response");
});

// Console Output when GET / is requested:
// A
// B
```

Passing Data Between Middleware

Middleware functions can attach data to the req object, which becomes available to the next middleware and the final route handler.

```
const attachUser = (req, res, next) => {
  req.user = { name: "Pratik", role: "Instructor" };
  next();
};

app.get("/profile", attachUser, (req, res) => {
  res.send(`Welcome ${req.user.name}`);
});
```

Common Errors

Error	Reason	Solution
Request hangs	One middleware in the chain did not call next()	Ensure every middleware calls next() or sends a response
Response already sent error	next() called after res.send() was already used	Only call next() when NOT sending a response in that middleware
Middleware runs on unintended routes	Middleware applied globally using app.use() instead of route-level	Apply middleware only to the specific route where needed

Interview Questions

What is Middleware Chaining?

Middleware Chaining is the execution of multiple middleware functions in sequence for the same request, where each one calls next() to pass control forward.

What is the difference between global and route-level middleware?

Global middleware (app.use()) runs for every request, while route-level middleware runs only for the specific route it is attached to.

What happens if a middleware in the chain does not call next()?

The request-response cycle stops there, and the client never receives a response — the request hangs.

Can middleware share data with the next middleware?

Yes — middleware can attach custom properties to the req object (e.g., req.user), which subsequent middleware and route handlers can access.

Summary

- Middleware Chaining executes multiple middleware functions in sequence for a single request.
- Each middleware must call next() to pass control to the following middleware or route handler.
- Middleware can be applied globally (app.use()) or at the route level for specific endpoints.
- Data can be passed between middleware functions by attaching properties to the req object.

Lecture 18

Error Handling in Express

Learning Objectives

- Understand what Error Handling is and why it matters
- Use try-catch to handle synchronous and asynchronous errors
- Handle requests to invalid or undefined routes
- Create a Custom Error Handling Middleware
- Use appropriate HTTP Status Codes for error responses

Introduction

In real applications, things can go wrong — invalid data, missing records, failed database connections, or bugs in code. If these errors are not handled properly, the server may crash or send unclear responses to the client. Error Handling ensures the application responds gracefully and stays stable even when something goes wrong.

What is Error Handling?

Error Handling is the process of catching errors that occur during request processing and sending a proper, meaningful response to the client instead of letting the application crash.

Real-World Analogy — Restaurant

If a restaurant runs out of an ingredient for a dish, the waiter doesn't simply disappear — they politely inform the customer and suggest an alternative. Similarly, when a server encounters an error, it should not crash silently — it should respond with a clear, appropriate message.

Restaurant	Backend
Dish Unavailable	Error Occurs
Waiter Informs Customer	Error Middleware Sends Response
Suggests Alternative	Meaningful Error Message + Status Code

Why is Error Handling Important?

- Prevents the application from crashing unexpectedly
- Gives the client clear, meaningful feedback
- Makes debugging easier for developers
- Improves the overall stability and reliability of the backend
- Follows professional, industry-standard practices

Using try-catch

try-catch is used to catch errors inside a block of code, especially for asynchronous operations like database queries.


```
app.get("/students/:id", async (req, res) => {
  try {
    const student = await Student.findById(req.params.id);

    if (!student) {
      return res.status(404).send("Student Not Found");
    }

    res.json(student);
  } catch (error) {
    res.status(500).send("Server Error");
  }
});
```

How It Works

- Code that might fail is placed inside the try block
- If an error occurs, execution jumps to the catch block
- The catch block sends an appropriate error response instead of crashing the server

Handling Invalid Student ID (Practical)

```
let students = [
  { id: 1, name: "Pratik" },
  { id: 2, name: "Rahul" },
];

app.get("/students/:id", (req, res) => {
  const id = Number(req.params.id);
  const student = students.find((s) => s.id === id);

  if (!student) {
    return res.status(404).send("Student Not Found");
  }

  res.json(student);
});
```

Request: GET /students/99 (an ID that does not exist)

```
// Response: 404 Student Not Found
```

Handling Invalid Routes

When a client requests a URL that does not match any defined route, Express should respond with a clear 404 message instead of a default error page.

```
// Placed after all valid routes
app.use((req, res) => {
  res.status(404).send("Route Not Found");
});
```

This middleware must be defined last, after all other routes, so it only runs when no route has matched the request.

Custom Error Middleware

Express supports a special type of middleware for handling errors — it takes four parameters instead of three: (err, req, res, next).

```
const errorHandler = (err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({
    message: "Something Went Wrong",
    error: err.message,
  });
};

app.use(errorHandler);
```

Why Four Parameters?

Express identifies a middleware as an Error Handling Middleware specifically because it has four parameters. It must always be defined after all routes and other middleware.

Triggering the Error Middleware

```
app.get("/error-demo", (req, res, next) => {
  const err = new Error("Something Broke");
  next(err); // passes control to the error middleware
});

app.use(errorHandler);
```

When next() is called with an argument, Express skips all remaining regular middleware and routes, and passes control directly to the error handling middleware.

Complete Example

```
const express = require("express");
const app = express();

app.use(express.json());

let students = [
  { id: 1, name: "Pratik" },
  { id: 2, name: "Rahul" },
];

// Route with try-catch
app.get("/students/:id", (req, res, next) => {
  try {
    const id = Number(req.params.id);
    const student = students.find((s) => s.id === id);

    if (!student) {
```

```
        const err = new Error("Student Not Found");
        err.statusCode = 404;
        throw err;
    }

    res.json(student);
} catch (error) {
    next(error);
}
});

// 404 handler for undefined routes
app.use((req, res) => {
    res.status(404).send("Route Not Found");
});

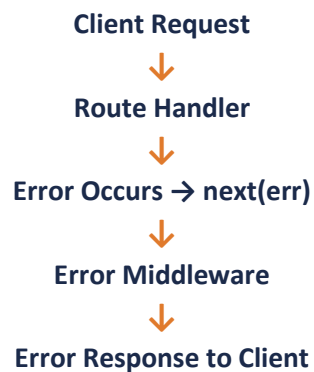
// Central error handling middleware
app.use((err, req, res, next) => {
    const statusCode = err.statusCode || 500;
    res.status(statusCode).json({
        message: err.message,
    });
});

app.listen(3000, () => console.log("Server Running"));
```

Status Codes Used in Error Handling

Status Code	Meaning	Example Use Case
400	Bad Request	Invalid data sent by client
401	Unauthorized	Missing or invalid authentication token
403	Forbidden	User authenticated but not permitted
404	Not Found	Requested resource or route does not exist
500	Internal Server Error	Unexpected server-side error

Error Handling Request Flow



Best Practices

- Always use try-catch for asynchronous database operations
- Never expose sensitive internal error details to the client
- Always send appropriate HTTP status codes with error responses
- Keep a single, centralized error handling middleware at the end of the application
- Log errors on the server for debugging, but keep client-facing messages simple

Common Mistakes

Mistake	Fix
Error middleware defined before other routes	Always place error handling middleware at the very end
Forgetting to call <code>next(err)</code> inside catch block	Use <code>next(error)</code> so Express routes it to the error middleware
Sending <code>res.send()</code> without a status code on errors	Always pair error responses with <code>res.status(code)</code>

Interview Questions

What is Error Handling in Express?

Error Handling is the process of catching and managing errors during request processing so the application responds gracefully instead of crashing.

How does Express identify an Error Handling Middleware?

By its four parameters — (`err`, `req`, `res`, `next`) — instead of the usual three.

What is the purpose of `next(err)`?

Calling `next()` with an argument tells Express to skip regular middleware and routes and pass control directly to the error handling middleware.

How do you handle requests to undefined routes?

By adding a catch-all middleware after all defined routes that responds with a 404 status.

Summary

- Error Handling prevents the application from crashing and gives clients meaningful feedback.
- try-catch is used to handle errors in synchronous and asynchronous route logic.
- A catch-all middleware handles requests to undefined routes with a 404 response.
- Custom Error Middleware uses four parameters — (`err`, `req`, `res`, `next`) — and must be defined last.
- Proper status codes (400, 401, 403, 404, 500) make error responses clear and professional.